

Práctica Evaluable II

Nexus Gate – Agencia de viajes gamer

Ana Cascone Hernández
Programación Web
Grado en Ingeniería Informática

Curso 2025–2026

Índice

1. Introducción	2
2. Estructura y organización del proyecto	2
3. Base de datos	2
4. Proceso de desarrollo	3
4.1. Estructura de las tablas e <code>init.sql</code>	3
4.2. <code>configuracion.inc.php</code>	3
4.3. Clases <code>DatosObject</code> y <code>Usuario</code>	3
4.4. <code>cabecera.inc.php</code> y <code>pie.inc.php</code>	3
4.5. Autenticación	3
4.6. Clases <code>Viaje</code> y <code>Pais</code>	4
4.7. Adaptación de HTML a PHP	4
4.8. <code>menu.inc.php</code> y <code>viajes.php</code>	5
4.9. <code>viaje.php</code>	5
4.10. <code>viajes_buscados.php</code>	5
4.11. Panel de administración	5
4.12. Carrusel de <code>index.php</code>	6
5. Validación de formularios	6
6. Conclusiones y dificultades	6
7. Posibles mejoras	6
8. Uso de la inteligencia artificial	7

1. Introducción

Esta práctica es la continuación de la anterior, en la que construí la web estática de *Nexus Gate*, una agencia de viajes con temática gamer. En aquella primera parte no había nada de lógica: solo HTML y CSS. El objetivo ahora ha sido dar vida a esa web añadiendo PHP, una base de datos MySQL y validación con JavaScript, manteniendo en todo momento la estética que ya tenía.

A grandes rasgos, el resultado permite registrarse e iniciar sesión, ver un listado de viajes paginado, filtrarlos por país, consultar la ficha de cada viaje, buscar por destino y fecha, y administrar los viajes (crear, editar y borrar) desde un panel reservado al administrador.

2. Estructura y organización del proyecto

Antes de escribir nada pensé cómo organizar los ficheros para que el proyecto fuera claro y cómodo de mantener. La estructura quedó así:

- `classes/`: las clases (`DatosObject`, `Usuario`, `Viaje`, `Pais`).
- `includes/`: piezas reutilizables (`configuracion.inc.php`, `cabecera.inc.php`, `pie.inc.php`, `menu.inc.php`, `sesion.inc.php`).
- `css/`, `js/`, `imagenes/`: ficheros de estilos, scripts e imágenes.
- `admin/`: el panel de administración.
- En la raíz, las páginas que ve el usuario (`index.php`, `viajes.php`, etc.).

Algunas decisiones que tomé y conviene explicar:

- **Prefijo `pe2_` en las tablas.** Comparto la misma base de datos con otros ejercicios de la asignatura, así que puse el prefijo `pe2_` a todas las tablas (`pe2_usuarios`, `pe2_paises`, `pe2_viajes`) para no mezclarlas.
- **Ficheros `.inc.php`.** El profesor usa la terminación `.inc`, pero un fichero `.inc` sin `.php` Apache lo sirve como texto plano, con lo que se vería el código fuente. Usar `.inc.php` es más seguro, porque el servidor lo ejecuta como PHP en lugar de mostrarlo.
- **Sin `?>` al final.** En los ficheros que son únicamente PHP no cierro la etiqueta, para evitar que un espacio o salto de línea de más se envíe al navegador y provoque errores (por ejemplo, al iniciar sesiones o enviar cabeceras).
- **Uso de `__DIR__`.** En los `require` utilizo `__DIR__`, que devuelve la ruta real de la carpeta del fichero actual. Así construyo rutas absolutas y los `include` funcionan igual sin importar desde qué carpeta se llamen.

3. Base de datos

Toda la estructura de la base de datos está en `init.sql`, que funciona como un guion de inicialización. Lo cargo con:

```
USE dbanach2173_pw2526;  
SOURCE init.sql;
```

`SOURCE` simplemente ejecuta una a una las órdenes SQL del fichero. Lo escribí de forma que se pueda volver a ejecutar siempre que quiera: primero borra las tablas si existen, después las crea de nuevo y por último las rellena con el usuario administrador y unos viajes de ejemplo. De este

modo, si en algún momento dejaba la base de datos hecha un lío probando cosas, podía resetearla al estado inicial volviendo a lanzar el mismo fichero.

Hay tres tablas: `pe2_usuarios`, `pe2_paises` y `pe2_viajes`. Decidí sacar los países a su propia tabla y que cada viaje guarde solo su `pais_id` (una clave ajena que apunta a `pe2_paises`). Así evito inconsistencias al escribir el nombre del país y el menú por continentes sale directo de esa tabla.

Las contraseñas no se guardan en claro: utilizo `password_hash()` al darse de alta y `password_verify()` al iniciar sesión, de modo que en la tabla solo queda el hash.

4. Proceso de desarrollo

4.1. Estructura de las tablas e `init.sql`

Empecé pensando qué columnas necesitaba cada tabla. Los datos de `pe2_usuarios` salen del propio formulario de registro (nombre, usuario, email, contraseña y fecha de nacimiento), más un campo `tipo` para distinguir al administrador. Creé el `init.sql` como base de datos «inicial» según lo explicado en el apartado anterior.

4.2. `configuracion.inc.php`

Después hice el fichero de configuración, que solo contiene constantes: los datos de conexión, el tamaño de página para la paginación y los nombres de las tablas. La idea es tener en un único sitio todo lo que podría cambiar, en vez de repetirlo por el código.

4.3. Clases `DatosObject` y `Usuario`

`DatosObject` es una clase abstracta de la que heredan las demás. Reúne lo común a todas las tablas: conectar y desconectar con PDO, guardar los datos de una fila y un método `devolverValor()` para leer un campo. `Usuario` hereda de ella y añade el login, el alta y las comprobaciones de duplicados.

4.4. `cabecera.inc.php` y `pie.inc.php`

Como la cabecera y el pie se repetían en todas las páginas, los saqué a dos ficheros para reutilizarlos y facilitar el mantenimiento. La cabecera usa varias variables que cada página define antes de incluirla:

- `$titulo`: el título del `<head>`.
- `$cssPagina`: la hoja de estilos propia de la página (además de la base y la responsive).
- `$paginaActiva`: en qué página estamos, para resaltar la pestaña del menú.
- `$bodyClass`: si el `<body>` necesita una clase especial (por ejemplo, para el fondo).
- `$mostrarLogin`: si la página debe mostrar el login en la cabecera (páginas como `registro.php` no lo necesitan).
- `$jsPagina`: los archivos JavaScript necesarios para esa página.

4.5. Autenticación

A continuación hice todo lo necesario para que el usuario pudiera darse de alta y entrar, apoyándome en la clase `Usuario`:

- **login.php**: recibe por POST el usuario y la contraseña del formulario de la cabecera y llama a `Usuario::comprobarLogin()`. Si las credenciales son correctas, guarda en la sesión el nombre de usuario y el tipo (que es justo lo que necesita la cabecera para mostrar al usuario y, si es administrador, el enlace al panel); si no, guarda un mensaje de error. En ambos casos vuelve al `index.php`.
- **logout.php**: vacía y destruye la sesión y vuelve al inicio.
- **registro.php**: es una página que se procesa a sí misma. Al enviar el formulario, comprueba en el servidor que no haya campos vacíos y que el usuario y el email no estén ya registrados (con `existeUsuario` y `existeEmail`). Si todo es correcto, da de alta al usuario con `insertarUsuario` (que guarda la contraseña con `password_hash`) y redirige a `alta_ok.php`; si hay errores, vuelve a mostrar el formulario con los mensajes encima.
- **alta_ok.php**: página de confirmación que se muestra tras un registro correcto.

4.6. Clases Viaje y Pais

`Pais` solo necesita un método, `obtenerPaises()`, que devuelve todos los países ordenados por continente y nombre (para el menú lateral y para el desplegable del administrador).

`Viaje` concentra todas las consultas de viajes, todas con el JOIN a países para traer también el nombre del país y el continente:

- `obtenerViajes($inicio, $numero)` y `contarViajes()`: el listado paginado y el total de viajes (para saber cuántas páginas hay).
- `obtenerViajesPorPais()` y `contarViajesPorPais()`: lo mismo, pero filtrando por un país, para que la paginación también funcione al filtrar.
- `obtenerViaje($id)`: un viaje concreto.
- `viajesPorPais($paisId)`: los viajes de un país, para el bloque «más viajes en este país».
- `buscarViajes($texto, $fecha)`: la consulta del buscador.
- `insertarViaje`, `actualizarViaje` y `eliminarViaje`: el CRUD del administrador.
- `obtenerViajesCarrusel()`: devuelve los primeros viajes para el carrusel.

En estas consultas uso alias de tabla para que las JOIN queden más legibles. En lugar de escribir el nombre completo (que al venir de una constante obliga a concatenar una y otra vez):

```
'SELECT ' . TABLA_VIAJES . '.*, ' . TABLA_PAISES . '.nombre AS pais, ' . TABLA_PAISES
    . '.continente
FROM ' . TABLA_VIAJES . '
JOIN ' . TABLA_PAISES . ' ON ' . TABLA_VIAJES . '.pais_id = ' . TABLA_PAISES . '.id'
```

uso algo más sencillo y visual:

```
'SELECT v.*, p.nombre AS pais, p.continente
FROM ' . TABLA_VIAJES . ' v
JOIN ' . TABLA_PAISES . ' p ON v.pais_id = p.id'
```

4.7. Adaptación de HTML a PHP

Pasé las páginas de la práctica anterior a `.php`, cambiando los enlaces de `.html` a `.php` y adaptándolas para reutilizar la cabecera y el pie:

- **index.php**: de momento dejé libre la parte del carrusel para más adelante.

- `ofertas.php` y `viajes_grupo.php`.
- `sugerencias.php` y `contacto.php`: las comprobaciones del formulario ahora las hace JavaScript, aunque el botón de enviar sigue sin tener lógica de servidor.

También añadí los CSS correspondientes para que `cabecera.inc.php` los cargara.

4.8. `menu.inc.php` y `viajes.php`

En `viajes.php` se lee la página actual de la URL, se calcula desde qué viaje empezar y, si se ha indicado un país, se filtra por él; al final se calcula el número total de páginas para la paginación. El `menu.inc.php` consulta los países con `Pais::obtenerPaíses()`, que los devuelve ya ordenados por continente, así que basta con un `foreach` para pintarlos agrupados.

A partir de aquí ya tengo la rejilla de viajes y puedo seleccionar un país para ver solo los suyos. He puesto pocos viajes por país, salvo Japón, que tiene muchos para poder comprobar que la paginación también funciona al filtrar por país.

4.9. `viaje.php`

Antes tuve que cambiar las tarjetas para que todas enlazaran al viaje individual, usando `<a href="viaje.php?id=..."` con el `id` del viaje (leído con `devolverValor`). Mantuve una estructura parecida a la de `viaje_ciudad.html` de la práctica anterior, pero ahora los datos se consultan con `devolverValor`. Para la descripción usé `nl2br()`, que convierte los saltos de línea de la base de datos en etiquetas `<br>`.

4.10. `viajes_buscados.php`

Recibe por `GET` los datos del buscador de `index.php` (`buscar_destino` y `buscar_fecha`), llama a `Viaje::buscarViajes()` para obtener los viajes cuyo destino o país contengan el texto y cuya fecha indicada esté entre la de inicio y la de fin, y los muestra con el mismo diseño de tarjetas que `viajes.php`, enlazando cada una con su ficha. Si no hay resultados, avisa de que no se ha encontrado ningún viaje. Por ejemplo, buscando «Japón» y «2026-09-11» salen los viajes a Tokio y a Osaka.

4.11. Panel de administración

El CRUD de los viajes vive en la clase `Viaje` (`insertarViaje`, `actualizarViaje` y `eliminarViaje`), todos con consultas preparadas. El panel está formado por:

- `admin/index.php`: lista todos los viajes en una tabla con enlaces para editar y eliminar.
- `admin/crear.php`: formulario para dar de alta un viaje; el país se elige de un desplegable que se rellena con `Pais::obtenerPaíses()`.
- `admin/editar.php`: el mismo formulario, pero con los datos del viaje ya cargados, que llama a `actualizarViaje`.
- `admin/eliminar.php`: borra el viaje indicado y vuelve al listado.

Todas estas páginas incluyen `sesion.inc.php` al principio, que comprueba que hay un usuario logueado y que es administrador; si no, redirige. Además modifiqué la cabecera para que solo aparezca el enlace a `admin/index.php` si el usuario es administrador, y creé la variable `$rutaBase` para que los ficheros dentro de `admin/` accedan correctamente a los CSS y a los enlaces.

4.12. Carrusel de index.php

Para el carrusel cojo los 6 primeros viajes. Creé el método `Viaje::obtenerViajesCarrusel()` para que quedara más semántico. En `index.php` un `foreach` genera el HTML de esos viajes, cada uno en un `<article class="slide">` que por defecto está oculto. En `carrusel.js` gestiono cuál es el slide activo con una variable `slideActual` y `classList.add/remove`.

5. Validación de formularios

Algo que debo destacar de la validación de formularios es que algunas comprobaciones tienen que hacerse en PHP: JavaScript, por ejemplo, no puede consultar la base de datos para saber si un usuario ya existe.

JavaScript (en cada formulario): comprueba que no haya campos vacíos y los formatos que antes hacía el HTML: email con formato válido, contraseña con longitud mínima y fecha con el formato correcto. En los formularios de edición y creación de viajes del administrador, además, que el precio sea un número y que la fecha de fin no sea anterior a la de inicio.

PHP: lo que JavaScript no puede saber: duplicados con `existeUsuario` y `existeEmail`, que el login sea correcto y, por seguridad, una nueva comprobación de que los campos no estén vacíos (porque el JavaScript se puede saltar).

El flujo general es:

1. El usuario pulsa «Enviar».
2. JavaScript valida.
3. Si hay errores, se muestran encima del formulario y no se envía; si no, se envía a PHP.
4. PHP vuelve a validar la seguridad básica.
5. PHP consulta la base de datos si hace falta.
6. Se inserta, se modifica o se inicia sesión.

El formulario de login no lleva validación de JavaScript: al estar en la cabecera puede aparecer en cualquier página, y las únicas comprobaciones realmente imprescindibles para él son las que dependen de la base de datos.

6. Conclusiones y dificultades

Lo más difícil fue pensar desde el principio una estructura correcta para el proyecto. Intenté prever todos los ficheros que iban a hacer falta y una forma de organizarlos que resultara clara y cómoda.

Partía con algo de ventaja, porque estoy cursando Sistemas de Información Basados en Web y hace unas semanas hice una práctica muy parecida, aunque con Twig y Docker; por eso mis mayores dificultades fueron la base de datos y la reutilización de código en PHP. Por lo demás, ha sido una práctica muy interesante y útil.

7. Posibles mejoras

- Más seguridad, como comprobar que la imagen existe realmente al crear un viaje.
- Actualmente no se puede crear un viaje a un país que no exista. Se podría añadir una página u opción para dar de alta nuevos países en `pe2_paises`.

8. Uso de la inteligencia artificial

A continuación detallo cada uso de IA generativa. En todos los casos la herramienta utilizada fue **Claude (Anthropic)**. Los prompts se presentan reconstruidos a partir de las consultas realizadas, ya que algunas se hicieron en conversaciones puntuales sin guardar el texto literal.

Uso 1: datos de ejemplo para los INSERT

- **Necesidad:** rellenar la tabla de viajes con datos de prueba realistas y suficientes para comprobar la paginación.
- **IA usada:** Claude.
- **Prompt e iteraciones:** pedí que generara varios viajes de ejemplo (destino, descripción, frase, precio y fechas) repartidos entre los países. La mayoría eran los que utilicé en la práctica anterior.
- **Solución y explicación:** devolvió las sentencias `INSERT INTO pe2_viajes (...)` con los datos. Son simplemente filas de datos; la estructura de la tabla y las claves ajenas las decidí yo.
- **Evaluación:** correcta y útil; me ahorró tiempo en una parte sin dificultad técnica. Revisé que los `pais_id` correspondieran con los países correctos.

Uso 2: reorganización de los CSS

- **Necesidad:** los CSS estaban desordenados y quería una estructura más cómoda.
- **IA usada:** Claude.
- **Prompt e iteraciones:** le pasé mis hojas de estilo y pedí ayuda para reorganizarlas y añadir los estilos de las clases nuevas (mensajes de error y usuario activo).
- **Solución y explicación:** reordenó y añadió reglas reutilizando mis variables de color. Es CSS, sin lógica; entiendo cada regla.
- **Evaluación:** correcta y acorde a la estética que ya tenía.

Uso 3: dudas de base de datos y consejo de los alias

- **Necesidad:** la base de datos es la parte que menos controlo, así que me apoyé en la IA de forma general: dudas sobre cómo plantear las consultas, el uso de PDO con sentencias preparadas, los JOIN y consejos de organización como el uso de alias.
- **IA usada:** Claude.
- **Prompt e iteraciones:** varias consultas a lo largo del desarrollo sobre cómo hacer y estructurar las consultas a la base de datos (no puntuales, sino según me iban surgiendo dudas).
- **Solución y explicación:** me ayudó a entender cómo montar las consultas con `prepare/bindValue`, a hacer los JOIN con países y a usar alias para que quedaran legibles. Todo lo apliqué entendiéndolo.
- **Evaluación:** explicación correcta y clara; mejora la legibilidad de mis consultas.

Uso 4: páginas del administrador (HTML/PHP repetitivo)

- **Necesidad:** acelerar la parte de HTML/CSS (ya evaluada en la práctica anterior) sin perder la estética, sobre todo en formularios repetitivos del panel como `crear.php` y `editar.php`.
- **IA usada:** Claude.
- **Prompt e iteraciones:** pedí ayuda para montar los formularios de crear y editar viaje conectados con los métodos de la clase `Viaje`.
- **Solución y explicación:** generó los formularios con el desplegable de países y el envío por POST a los métodos `insertarViaje` y `actualizarViaje`. La lógica de las clases la entiendo y puedo explicarla.
- **Evaluación:** correcta; sigue el patrón del resto del proyecto. Comprobé que los nombres de los campos coincidían con los del `$_POST`.

Uso 5: `htmlspecialchars`

- **Necesidad:** mostrar datos de la base de datos en el HTML de forma segura.
- **IA usada:** Claude.
- **Prompt e iteraciones:** pregunté cómo evitar problemas al imprimir texto que viene de la base de datos.
- **Solución y explicación:** me aconsejó usar `htmlspecialchars()` para que caracteres especiales no rompan el HTML. Lo apliqué en todas las salidas.
- **Evaluación:** buena práctica, correcta y sencilla de justificar.

Uso 6: validaciones en JavaScript

- **Necesidad:** validar los formularios con expresiones regulares y comprobar fechas correctamente.
- **IA usada:** Claude.
- **Prompt e iteraciones:** pedí ayuda con las expresiones regulares (email, usuario, fecha) y con la comprobación de que una fecha es válida.
- **Solución y explicación:** entre otras cosas, propuso comprobar la validez de la fecha así:

```
const fechaObj = new Date(fecha.value);
if (isNaN(fechaObj.getTime())) {
    errores.push('La fecha no es valida.');
```

Crea un objeto `Date` y, si no es una fecha real, `getTime()` devuelve `NaN`.

- **Evaluación:** correcta. Tuve que ajustar a qué formulario se enganchaba la validación (recorrer los formularios del `main`), porque inicialmente se aplicaba al de login de la cabecera.

Uso 7: redacción de la memoria

- **Necesidad:** redactar esta memoria de forma ordenada a partir de mis propias notas.
- **IA usada:** Claude.

- **Prompt e iteraciones:** le pasé mis notas del proceso y pedí que las redactara en formato LaTeX.
- **Solución y explicación:** estructuró y redactó el documento siguiendo mis notas. El contenido, las decisiones y el desarrollo son míos; la ayuda fue de redacción y formato.
- **Evaluación:** útil para presentar el trabajo de forma clara; revisé que no hubiera añadido o modificado nada relevante.